



Inheritance

1. Concept
2. Terminologies
3. Implementing inheritance
4. Type of inheritance
5. Protected keyword
6. Diamond ring problem and Virtual inheritance

- Purpose of inheritance
 - Reuse existing entities to create more specialized forms of that entity
 - Existing entities are not MODIFIED, they are EXTENDED
 - No need to recreate and test the existing entities. Just the extensions are created and tested. This saves lot of time in development of the software.
- IS-a relationship is created

- Base class. Also called as Super class or Parent class
- Derived class. Also called as Sub class or Child class
- IS-A Relationship and its direction
- Arrow notation for IS-A relationship
- Inheritance hierarchy
- Generalization to specialization
- Principles of substitution

- Syntax for extending a class to create its more specialized type
 - `class Employee: public class Person`
`{`
`}`
 - Order of creation of object: From base to derived
 - Order of execution of constructor : From base to derived
 - Order of execution of destruction: From derived to base
 - Role of parameterized constructor of derived class
 - Member initialization list: MIL

4. Type of inheritance

- Single level inheritance
- Multi level inheritance
- Hierarchical inheritance
- Multiple inheritance
- Hybrid inheritance
- private inheritance
- protected inheritance

- Protected access specifier
- Accessible in own class as well as in all other classes derived from it directly or indirectly at multiple levels
- Accessibility is more than private but less than public
- Even inheritance can be protected but not recommended

- One class gets inherited through more than one routes
- Data members and member functions of that class get inherited multiple times. This leads to ambiguity error.
- This can be avoided using virtual inheritance
- Multiple inheritance is NOT recommended as it can lead to diamond ring problem in future.
- Next generation object oriented languages DO NOT support multiple inheritance